

Chap4 处理器体系结构

1、RISC/CISC:

RISC: 指令长度固定

CISC (X86-64、Y86-64 为 X86-64 子集): 指令长度可变

2、Y86-64 指令集

(1) 指令编码长度 1 到 10 个字节

(2) 编码结构

① $\text{icode} + \text{ifun} = 1$ 字节, 所有指令都有

② 寄存器标识符字节 (rA/rB/rA 和 rB) = 1 字节, Y86-64 共 15 个 64 位程序寄存器, 标识符 $0x0-0xE$ 对应寄存器文件中相应寄存器, $0xF$ 代表无寄存器

1)

③ 常数 (立即数 V 、地址偏移 D 、目的地址 Dest) = 4 字节——>填充 0 扩展为 8 字节, 按小端序填入指令编码

字节	0	1	2	3	4	5	6	7	8	9
halt	0	0								
nop	1	0								
rrmovq rA, rB	2	0	rA	rB						
irmovq V, rB	3	0	F	rB					V	
rmmovq rA, D(rB)	4	0	rA	rB					D	
mrmmovq D(rB), rA	5	0	rA	rB					D	
OPq rA, rB	6	fn	rA	rB						
jXX Dest	7	fn							Dest	
cmovXX rA, rB	2	fn	rA	rB						
call Dest	8	0							Dest	
ret	9	0								
pushq rA	A	0	rA	F						
popq rA	B	0	rA	F						

3、顺序实现

取指、译码、执行、访存、写回、更新 PC

1) 取指:

- 从指令存储器读取指令
- $\text{valC} = V/D/\text{Dest}$
- $\text{valP} = \text{PC} + \text{指令长度}$

2) 译码: $\text{valA} = R[rA]$, $\text{valB} = R[rB]$

3) 执行: valE 、CC 条件码 更新

4) 访存: $\text{valM} = M[]$

5) 写回: valM 、 valE 写回寄存器

6) 更新 PC: $PC = valP$

4、各指令 6 阶段微指令

阶段	OPq rA, rB	rrmovq rA, rB	irmovqV, rB
取指	icode; ifun $\leftarrow M_1[PC]$ rA; rB $\leftarrow M_1[PC+1]$ valP $\leftarrow PC+2$	icode; ifun $\leftarrow M_1[PC]$ rA; rB $\leftarrow M_1[PC+1]$ valP $\leftarrow PC+2$	icode; ifun $\leftarrow M_1[PC]$ rA; rB $\leftarrow M_1[PC+1]$ valC $\leftarrow M_8[PC+2]$ valP $\leftarrow PC+10$
译码	valA $\leftarrow R[rA]$ valB $\leftarrow R[rB]$	valA $\leftarrow R[rA]$	
执行	valE $\leftarrow valB \text{ OP } valA$ Set CC	valE $\leftarrow 0 + valA$	valE $\leftarrow 0 + valC$
访存			
写回	$R[rB] \leftarrow valE$	$R[rB] \leftarrow valE$	$R[rB] \leftarrow valE$
更新 PC	$PC \leftarrow valP$	$PC \leftarrow valP$	$PC \leftarrow valP$

阶段	jXX Dest	call Dest	ret
取指	icode; ifun $\leftarrow M_1[PC]$ valC $\leftarrow M_8[PC+1]$ valP $\leftarrow PC+9$	icode; ifun $\leftarrow M_1[PC]$ valC $\leftarrow M_8[PC+1]$ valP $\leftarrow PC+9$	icode; ifun $\leftarrow M_1[PC]$ valP $\leftarrow PC+1$
译码		valB $\leftarrow R[\%rsp]$	valA $\leftarrow R[\%rsp]$ valB $\leftarrow R[\%rsp]$
执行	Cnd $\leftarrow \text{Cond}(CC, \text{ifun})$	valE $\leftarrow valB + (-8)$	valE $\leftarrow valB + 8$
访存		$M_8[valE] \leftarrow valP$	valM $\leftarrow M_8[valA]$
写回		$R[\%rsp] \leftarrow valE$	$R[\%rsp] \leftarrow valE$
更新 PC	$PC \leftarrow \text{Cnd?valC; valP}$	$PC \leftarrow valC$	$PC \leftarrow valM$

阶段	rrmovq rA, D(rB)	rrmovq D(rB), rA
取指	icode; ifun $\leftarrow M_1[PC]$ rA; rB $\leftarrow M_1[PC+1]$ valC $\leftarrow M_8[PC+2]$ valP $\leftarrow PC+10$	icode; ifun $\leftarrow M_1[PC]$ rA; rB $\leftarrow M_1[PC+1]$ valC $\leftarrow M_8[PC+2]$ valP $\leftarrow PC+10$
译码	valA $\leftarrow R[rA]$ valB $\leftarrow R[rB]$	valB $\leftarrow R[rB]$
执行	valE $\leftarrow valB + valC$	valE $\leftarrow valB + valC$
访存	$M_8[valE] \leftarrow valA$	valE $\leftarrow M_8[valE]$
写回		$R[rA] \leftarrow valM$
更新 PC	$PC \leftarrow valP$	$PC \leftarrow valP$

阶段	pushq rA	popq rA
取指	<code>icode:ifun<-M1[PC]</code> <code>rA:rB<-M1[PC+1]</code> <code>valP<-PC+2</code>	<code>icode:ifun<-M1[PC]</code> <code>rA:rB<-M1[PC+1]</code> <code>valP<-PC+2</code>
译码	<code>valA<-R[rA]</code> <code>valB<-R[%rsp]</code>	<code>valA<-R[%rsp]</code> <code>valB<-R[%rsp]</code>
执行	<code>valE<-valB-8</code>	<code>valE<-valB+8</code>
访存	<code>M8[valE]<-valA</code>	<code>valM<-M8[valA]</code>
阶段	pushq rA	popq rA
写回	<code>R[%rsp]<-valE</code>	<code>R[%rsp]<-valE</code> <code>R[rA]<-valM</code>
PC更新	<code>PC<-valP</code>	<code>PC<-valP</code>

valA 读内存 valB 操作栈指针

入栈先减小栈指针再存数，出栈先取数再增加栈指针

4、流水线的实现基础

1. 参照 Y86-64 流水线 CPU 的实现，说明流水线如何工作。

答：

流水线化的系统，待执行的任务被划分成若干个独立的阶段，将处理器的硬件 也组织成若干个单元，让各个独立的任务阶段在不同的硬件单元上一次执行， 从而使多个任务并行操作。

结合案例：如 Y86-64 将指令执行分为取指、译码、执行、访存、 写回 5 个阶段，通过在每个阶段插入流水线寄存器，利用时钟信号控制流水线 的时序和操作，理想情况下可实现 5 条指令的同时运行。

6、流水线实现高级技术

1. Y86-64 流水线 CPU 中的冒险的种类与处理方法。

答:

(1) 数据冒险: 指令使用寄存器 R 为目的, 瞬时之后使用 R 寄存器为源。

处理方法有:

① 暂停: 通过在执行阶段插入气泡 (bubble/nop), 使得当前指令执行暂停在译码阶段;

② 数据转发: 增加 valM/valE 的旁路路径, 直接送到译码阶段;

(2) 加载使用冒险: 指令暂停在取指和译码阶段, 在执行阶段插入气泡 (bubble/nop)

(3) 控制冒险: 分支预测错误: 在条件为真的地址 target 处的两条指令分别插入 1 个 bubble。

ret: 在 ret 后插入 3 个 bubble。

7、处理器的性能

CPI: Cycle Per Instruction。

1. CPI 的计算:

C: 时钟周期

I: 执行完成的指令数

B: 插入的气泡个数 ($C = I + B$)

$$CPI = C/I = (I+B)/I = 1.0 + B/I$$

$$B/I = LP + MP + RP$$

■ LP: 由加载/使用冒险停顿产生的处罚

▪ 加载指令的比例	0.25
▪ 加载指令需要停顿的比例	0.20
▪ 每次插入气泡的数量	1

$$\Rightarrow LP = 0.25 * 0.20 * 1 = 0.05$$

■ MP: 由错误的分支预测产生的处罚

▪ 条件转移指令的比例	0.20
▪ 条件转移预测错误的比例	0.40
▪ 每次插入气泡的数量	2

$$\Rightarrow MP = 0.20 * 0.40 * 2 = 0.16$$

■ RP: 由ret指令产生的处罚

▪ 返回指令站的比例	0.02
▪ 每次插入的气泡数量	3

$$\Rightarrow RP = 0.02 * 3 = 0.06$$

处罚造成的影响 (三种处罚的总和) $0.05 + 0.16 + 0.06 = 0.27$

$$\rightarrow CPI = 1.27$$

2. 补充: 超标量 $CPI > 1$ 。

Typical Values

